

Identity & Access Management

for Multi-Cloud SaaS

Authentication and authorization flows, patterns, and design principles for SaaS platforms operating across AWS, Azure, and Google Cloud.

Coverage

- Foundational concepts: AuthN, AuthZ, federation, SSO
- Eight authentication flows, end-to-end — from password to passkey to workload identity
- Five authorization models — RBAC, ABAC, ReBAC, policy-as-code, just-in-time
- Federation patterns across AWS, Azure, and Google Cloud
- Tenant-aware identity in multi-tenant SaaS; cross-cloud workload identity
- Threat models, anti-patterns, and design principles

Contents

1	Foundational concepts	AuthN vs AuthZ, federation, SSO, tokens
2	The eight authentication flows	Password → OIDC → SAML → client-creds → workload identity → mTLS → passkeys
3	Token anatomy	ID, access, refresh — what's in each and how they differ
4	Federation — in detail	How SSO actually works; where the user lands; the broker pattern
5	Authorization models	RBAC, ABAC, ReBAC, policy-as-code, just-in-time
6	Multi-cloud identity	Workload identity federation across AWS, Azure, GCP
7	Tenant-aware identity in SaaS	tenant_id provenance, claim propagation, isolation
8	Other methods	SPIFFE/SPIRE, SCIM, CIBA, step-up, decentralized identity, network identity
9	Threat model & anti-patterns	What attackers target; what to never do
10	Design principles & decision guide	How to choose between flows; the one-page summary

How to read this reference

Each section is self-contained and follows the same depth and tone as the AWS, Azure, and Google Cloud reference volumes. Flows are illustrated as numbered step sequences, and trade-offs are explicit. Where vendor specifics matter, controls are mapped across the three major clouds. This is a working reference for architects and engineers, not a marketing or compliance overview.

1. Foundational concepts

The vocabulary of identity is precise. Confusing these terms is the most common cause of insecure designs.

Authentication vs. authorization

Authentication (AuthN) answers **who are you?** Proving identity. The output is a credential or token that asserts identity. Authorization (AuthZ) answers **what may you do?** Deciding what an authenticated identity is allowed to access. The output is an allow or deny decision. These are separate concerns, even when they happen in the same request, and a clean design treats them as separate layers — authenticate once, authorize everywhere.

Why this matters

The most common architectural mistake is conflating the two. A valid token proves a user is *authenticated*; it does not, on its own, prove they are *authorized* to read a particular tenant's data. Every API call must re-evaluate authorization against the resource being accessed, using the identity claims in the token — not trust the token's mere presence.

Federation and SSO

Federation means trusting another system to perform authentication for you. You consume an assertion from a trusted issuer rather than verifying credentials yourself. **Single sign-on (SSO)** is a user experience where one authentication unlocks many applications. SSO is usually implemented via federation, but they are not the same word — federation is the mechanism, SSO is the experience.

In a multi-tenant SaaS, federation is what lets each enterprise tenant keep using their own identity provider (Okta, Entra ID, Google Workspace, Ping, AWS IAM Identity Center) while your application sees a consistent identity model. The tenant's IdP remains the source of truth for who their users are; your platform inherits that trust through a signed assertion.

Identity providers, brokers, and consumers

Role	What it does	Examples
Identity Provider (IdP)	Authenticates users; issues signed assertions or tokens	Okta, Entra ID, Google Workspace, Ping, AWS IAM Identity Center, Auth0
Identity broker	Sits between many IdPs and your apps; normalizes tokens	Amazon Cognito, Auth0, Azure AD B2C, Google Identity Platform
Relying party / SP	Trusts assertions from the IdP/broker; consumes them to authenticate users	Your SaaS application, an API gateway, a managed service
Authorization server	Issues access tokens for protected resources (OAuth 2.0)	Cognito, Auth0, Okta, custom OAuth servers
Resource server	Validates access tokens; enforces authorization on resources	Your API, AWS APIs, Azure APIs, GCP APIs

Tokens, claims, and assertions

Token — a cryptographically protected artifact carrying identity and/or authorization information. The most common form is a JWT (JSON Web Token), a signed JSON document. **Claim** — a single statement inside a token (“the user’s email is X,” “the tenant_id is A”). **Assertion** — the SAML term for a signed XML document making claims about a user.

A JWT consists of three parts: **header** (algorithm and key ID), **payload** (claims), and **signature** (cryptographic proof from the issuer). The signature is the part that matters — it is what makes a token trustworthy. JWTs are not encrypted by default; anyone holding one can read its claims. **Never put secrets in a JWT.**

```
Header: { "alg": "RS256", "kid": "key-2025-01" }
Payload: { "iss": "https://auth.yoursaas.com",
  "sub": "user-12345",
  "aud": "api.yoursaas.com",
  "exp": 1735689600,
  "tenant_id": "A",
  "scope": "analytics:read dashboards:read",
  "acr": "mfa" }
Signature: RS256(header.payload, issuer_private_key)
```

2. The eight authentication flows

Each flow has its place. Choose deliberately by use case — there is no single right answer for an entire SaaS.

Flow A — Direct password authentication

The user enters a username and password into your application; your application verifies them against a user store and issues a session token. This is the legacy pattern and is increasingly rare in enterprise SaaS, but it remains common for direct-signup, consumer-style products.

Sequence

- User submits credentials over HTTPS to your application.
- Application looks up the user and verifies the password against a bcrypt/Argon2 hash.
- Application checks MFA factors (TOTP, push, WebAuthn) if enabled.
- On success, application issues a session token (cookie or JWT).
- Session is bound to a per-tenant context if multi-tenant.

Required disciplines

- **Password storage** — Argon2id or bcrypt, never plain or fast hashes; per-user salt; appropriate work factor.
- **MFA** — mandatory for any non-trivial access; WebAuthn/passkey preferred over TOTP, TOTP over SMS.
- **Breach detection** — check passwords against known-breach corpora at set and reset time.
- **Rate limiting and lockout** — protect against credential stuffing and brute force; lockout policies that resist denial-of-service.
- **Session security** — HttpOnly, Secure, SameSite=strict cookies; short lifetimes; rotation on privilege change.

When to use it

Only when federation is not viable — small tenants without their own IdP, or consumer-style sign-up. Enterprise tenants will demand federation, and rightly so. If you must support direct passwords, treat the user store as a high-value asset and invest in the disciplines above.

Flow B — OAuth 2.0 + OIDC (Authorization Code with PKCE)

The gold standard for browser and mobile applications. The Authorization Code flow with PKCE (Proof Key for Code Exchange) is now mandatory for public clients under OAuth 2.1, and is the foundation of every modern federated authentication design.

Sequence

1. App generates `code_verifier` (random) and `code_challenge` = SHA256(`code_verifier`)
2. App → IdP /authorize (`response_type=code`, `code_challenge`, `code_challenge_method=S256`)
3. IdP presents login UI; user authenticates (and completes MFA)
4. IdP → App callback URL (authorization code, one-time use, short-lived)
5. App → IdP /token (`code` + `code_verifier` + `client_id` [+ `client_secret` if confidential])
6. IdP → App (`id_token`, `access_token`, `refresh_token`)
7. App stores tokens securely; uses `access_token` as Bearer on API calls
8. When `access_token` expires: App → IdP /token with `refresh_token` → new `access_token`

Why PKCE matters

Without PKCE, an attacker who steals the authorization code from the redirect URL (via referer leaks, browser history, malicious extensions, or open redirects on the callback host) can exchange it for tokens. PKCE binds the code exchange to a secret only the original client knows — the `code_verifier`. The attacker has the code but not the verifier, and the exchange fails. **Use PKCE for all clients**, not just public ones; the additional cost is zero.

Token handling

- **Access tokens** should be minutes-lived (15–60 minutes). They are bearer tokens — anyone holding one can use it until it expires.
- **Refresh tokens** should be rotated on every use (each refresh issues a new one and revokes the old) and revocable by the user or an admin.
- **ID tokens** should never be sent to APIs as authorization. They are for the client to learn who the user is, nothing more.
- **Storage**: avoid `localStorage`; prefer `HttpOnly` cookies (with strict `SameSite`) or a backend-for-frontend pattern that keeps tokens out of the browser entirely.

Flow C — SAML 2.0 federation

The enterprise default for the last two decades. XML-based, more cumbersome than OIDC, but still dominant in enterprise federation. Any serious multi-tenant SaaS must support SAML, because many tenants' IdPs (or their compliance regimes) require it.

Sequence (SP-initiated)

1. User → App (no session)
2. App constructs SAML AuthnRequest, redirects browser to IdP SSO endpoint
3. IdP authenticates user (with MFA), evaluates conditional access
4. IdP signs a SAML assertion (XML) containing the user's identity and attributes
5. IdP → browser → POST signed assertion to App's Assertion Consumer Service (ACS) URL
6. App validates the XML signature, extracts attributes (incl. tenant_id), creates a session
7. User is logged in

Signature validation is everything

The most famous SAML exploits — XML Signature Wrapping, comment injection, signature stripping — all stem from libraries that did not validate the signed assertion strictly enough. Use a battle-tested library (one your security team has reviewed and that is still maintained), never roll your own SAML parsing, and verify: (1) signature is valid, (2) signer is the trusted IdP, (3) assertion has not been replayed (one-time NotOnOrAfter), (4) audience matches your SP, (5) conditions are met.

Operational warning

SAML metadata exchange and certificate rotation are common operational failures. Each tenant's IdP signs assertions with a certificate that expires; your platform must support multiple valid signing certificates per IdP, monitor expirations, and have a tested rotation process. SAML outages from expired certificates are routine and avoidable.

Flow D — OIDC federation (the modern enterprise default)

The same user experience as SAML, but JSON-based, simpler, and far easier to debug. This is what Okta-as-OIDC, Google Workspace, Azure AD as OIDC, and AWS IAM Identity Center as OIDC look like. New federations should default to OIDC unless the tenant requires SAML.

Sequence

1. User → App; App redirects to IdP /authorize (OAuth 2.0 + OIDC)
2. User authenticates at IdP + MFA + conditional access
3. IdP → App callback with authorization code (with PKCE)
4. App (or broker) → IdP /token to exchange the code
5. IdP returns ID token (JWT signed with its private key) + access token
6. App validates the ID token against the IdP's published JWKS (JSON Web Key Set)
7. App maps claims (incl. tenant_id) into your platform's session

JWT signature validation is far harder to get catastrophically wrong than SAML signature validation, which is one reason OIDC is winning. The library still matters — a famous historical bug allowed the alg: none header to bypass signature verification — but the protocol surface is much smaller.

Flow E — OAuth 2.0 Client Credentials (machine-to-machine)

For server-to-server API calls where no user is involved: partner integrations, your own backend services calling each other, third-party SaaS platforms connecting to your APIs. The access token represents a workload, not a user.

Sequence

1. Service has `client_id` + `client_secret` (stored in a secret manager, rotated)
2. Service → Authorization server `/token` (`grant_type=client_credentials`, `scope=...`)
3. Authorization server returns short-lived access token (no user identity claims)
4. Service uses access token as Bearer on API calls
5. Resource server validates the token's signature, audience, expiry, and scope

Replacing user roles with token scopes

Because no user is involved, authorization is driven by **scopes** bound to the client, not roles. A scope like `analytics:read:tenant-A` says “this client may read analytics for Tenant A and nothing else.” Scopes should be narrow, specific, and tenant-aware where appropriate. The client should request only the scopes it needs for the call (downscoping).

Secret rotation discipline

- Store client secrets in a secret manager (AWS Secrets Manager, Azure Key Vault, GCP Secret Manager) — never in code, config files, or environment variables that persist.
- Rotate on a schedule (90 days is a reasonable default) and on any suspected compromise.
- Use rotation that issues a new secret while keeping the old one valid for a short overlap window — prevents outage during rollover.
- Better: eliminate the secret entirely with workload identity federation (Flow F).

Flow F — Workload identity federation (the zero-secrets pattern)

The modern best practice for machine-to-cloud authentication, and the canonical answer to “how do CI/CD pipelines and cross-cloud workloads authenticate without storing credentials?” It is also how a workload in one cloud calls services in another.



Workload Identity Federation — zero stored secrets

1. The workload already has a verifiable identity (K8s SA token / GitHub OIDC token / Azure WI).
2. AWS STS verifies the JWT signature against the registered OIDC issuer (Web Identity).
3. STS returns temporary credentials scoped by IAM role; nothing long-lived ever exists on disk.

The principle: the workload already has a verifiable cryptographic identity (a Kubernetes service-account token, a GitHub Actions OIDC token, an Azure managed identity, a SPIFFE SVID). The target cloud is configured to **trust**

the issuer of that identity, and exchanges the verified identity for short-lived cloud credentials. No client secret exists anywhere.

The same pattern across clouds

Source	Target	Mechanism
GitHub Actions	AWS	OIDC trust + sts:AssumeRoleWithWebIdentity
GitHub Actions	GCP / Azure	Workload Identity Federation
Kubernetes pod	AWS	IRSA (IAM Roles for Service Accounts) via OIDC
Kubernetes pod	GCP	Workload Identity → GCP service account
Kubernetes pod	Azure	Azure Workload Identity (via OIDC)
Workload in cloud X	Cloud Y	OIDC trust between X's identity issuer and Y's IAM
On-prem / external	AWS	IAM Roles Anywhere (X.509-based)

Flow G — Mutual TLS (mTLS)

Both client and server present X.509 certificates and verify each other at the TLS layer — authentication happens before any application-layer request is processed. Used heavily for service mesh in Kubernetes, partner integrations with strict requirements, and IoT/edge devices.

Where mTLS earns its place

- **Service mesh (Istio, Linkerd)** — every pod-to-pod call is mutually authenticated and encrypted, with certificates issued and rotated by the mesh's CA (often SPIRE-backed). This is what “encrypted and authenticated everywhere inside the cluster” looks like.
- **Partner B2B integrations** — strong machine identity for high-value APIs (payment, healthcare). Certificate pinning prevents impersonation.
- **IoT and edge** — devices provisioned with a unique certificate at manufacture; the certificate is the device's identity for life.
- **Public APIs requiring exceptionally strong client identity** — used in financial, government, and regulated sectors.

Operational considerations

- **Certificate lifecycle** is the hard part — issuance, rotation, revocation, and expiry monitoring. Automate or use a mesh that automates it.
- **Trust store management** — keep the list of trusted CAs minimal and tightly controlled; one rogue CA compromises the whole estate.
- **Revocation** — OCSP or short certificate lifetimes; many mesh implementations rotate certs hourly, sidestepping revocation entirely.
- **Performance** — modest TLS handshake overhead, amortized by connection reuse; not a real concern at modern hardware.

Flow H — Passwordless / passkeys (WebAuthn, FIDO2)

The replacement for passwords plus SMS MFA. Cryptographic key pairs bound to the user's device, unlocked by biometric or PIN. This is the single highest-leverage upgrade an enterprise can make to its authentication posture, and it is rapidly becoming the default for serious security programs.

Sequence (login)

1. User → App; App requests authentication
2. App's server sends a random challenge to the browser
3. Browser/OS prompts user via authenticator (Touch ID, Windows Hello, hardware key)
4. Authenticator signs the challenge with the user's private key
 - private key never leaves the device
 - signature includes the origin (your domain) – phishing-resistant
5. App's server verifies the signature against the user's registered public key
6. User is authenticated

Why phishing-resistant

The signed challenge includes the **origin** (the actual domain the user is on). A phishing site at a look-alike domain cannot trick the user's authenticator into signing anything useful for the real site — the origin in the signature would be wrong. This single property defeats the most common credential-theft pattern on the internet today.

Practical recommendation

For a modern SaaS, the path is: **passkeys as the primary user authentication factor**, with traditional password + TOTP as a fallback only for legacy or unsupported devices. Most enterprise tenants are now in a position to require this through their own IdP — your job is to support it cleanly via federation and not undermine it with weaker fallbacks.

3. Token anatomy

The three tokens you will encounter constantly. Each has a distinct purpose; using one in place of another is a security bug.

ID TOKEN <i>OIDC</i>	ACCESS TOKEN <i>OAuth 2.0</i>	REFRESH TOKEN <i>OAuth 2.0</i>
Purpose Who the user is	Purpose What may be called	Purpose Get new access tokens
Audience Audience: your app	Audience Audience: your API	Audience Used only at /token
Typical claims sub, email, tenant_id name, groups	Typical claims scope, tenant_id client_id, exp	Typical claims opaque or rotated JWT, revocable

Standard JWT claims you will see

Claim	Meaning	Example / use
iss	Issuer — who minted the token	https://auth.yoursaas.com
sub	Subject — the user or workload identity	user-12345
aud	Audience — who the token is intended for	api.yoursaas.com
exp	Expiration time (Unix timestamp)	Validate; reject if past
iat	Issued at	Useful for replay windows
nbf	Not before	Token is invalid before this time
jti	Unique token ID	Enables revocation / replay detection
scope	OAuth scopes granted	analytics:read dashboards:read
tenant_id	Custom: which tenant the user/workload belongs to	Drives multi-tenant authorization
acr	Authentication Context Class — strength of auth	mfa, passkey — used for step-up
amr	Authentication Methods References	["pwd", "otp"] or ["hwk"]

Validation checklist — every token, every time

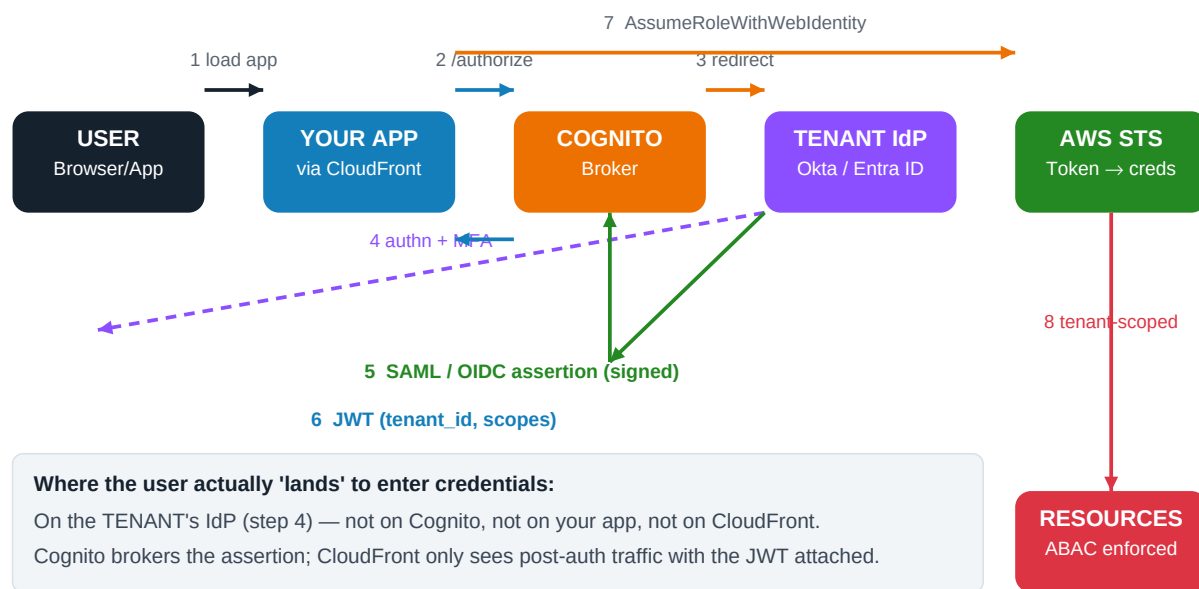
Token validation is mechanical but unforgiving — each of these checks closes a real attack class. Use a vetted library, but ensure all of these are happening:

- **Signature** — verify cryptographically against the issuer's public key (JWKS). Reject a `alg: none`; reject HS256 if you expect RS256.
- **Issuer** — `iss` matches the expected issuer URL exactly. No string-prefix matching.
- **Audience** — `aud` is your API, not someone else's. Audience confusion is a known cross-service vulnerability class.
- **Expiration** — `exp` is in the future, with a small leeway (60 seconds max) for clock skew.
- **Not before** — `nbf`, if present, is in the past.
- **Key ID** — `kid` in the header matches a key in the issuer's JWKS; rotate keys regularly.
- **Tenant context** — the `tenant_id` claim is present and matches the tenant the request is for. **Never derive `tenant_id` from user input.**
- **Token type** — you are validating the right kind of token for the use case (access tokens for API calls; not ID tokens).

4. Federation — how SSO actually works

The most common SaaS security design question: where does the user actually land to enter their credentials? The answer is rarely “your application.”

The federated authentication flow, end to end



The diagram above traces a Tenant A user signing in to a SaaS analytics dashboard, where Tenant A federates from their corporate Okta. The numbered steps correspond to the canonical SP-initiated SSO flow. Three observations are worth emphasizing:

- **The user lands on their tenant's IdP**, not on your application or on Cognito. This is by design — you never see the user's password.
- **Cognito (or whichever broker you use) is a translator**, not the authenticator. It converts the IdP's SAML/OIDC assertion into your platform's JWT.
- **The CDN (CloudFront / Front Door / Cloud CDN) does not handle the auth dance.** It sits in front of your application origin; auth redirects bounce directly to the IdP and broker endpoints. Only post-authentication application traffic flows through the CDN.

The broker pattern — why use Cognito (or Auth0, or Azure AD B2C) at all?

Without a broker, your application must directly accept SAML and OIDC tokens from every federated tenant — dozens or hundreds of different IdPs, each with its own quirks, certificate rotation schedule, and claim mapping. The broker collapses this surface: it speaks to every IdP, normalizes their assertions, and presents your application with a single, consistent token format.

Pattern	Pros	Cons	Choose when
Broker (Cognito / Auth0)	One token format for your app; per-tenant IdP mapping in one place; standard MFA fallback for direct-signup users	Broker becomes a critical dependency; vendor-specific quirks	Multi-tenant SaaS with many federated tenants; mix of federated and direct-signup tenants
Direct federation	Fewer hops; no broker to manage; lowest latency	App must handle multiple issuers and trust mappings; harder to add a new tenant	Small number of tenants; team has capacity to manage federation directly
IdP-of-record only (no SSO)	Simplest; full control	No enterprise SSO support; loss of large deals	Consumer / SMB only

Where the `tenant_id` comes from in a federated flow

This is one of the most critical and most often mishandled details. The `tenant_id` claim must come from a source you trust:

- **From the IdP assertion** (most robust) — the tenant's IdP sends `tenant_id` as a SAML attribute or OIDC claim. Your broker maps it into the token.
- **From the broker's app-client configuration** — each tenant has a dedicated app client or federated IdP entry in the broker; the broker's post-authentication trigger (e.g. a Cognito Lambda) injects the `tenant_id` based on which connection authenticated the user.
- **From a server-side lookup keyed by the IdP issuer** — your application receives a token from an IdP and looks up which tenant that IdP belongs to in a server-side mapping you control.

Critical security rule

Never derive `tenant_id` from anything the user sends in the request. Not from a URL parameter, not from a header, not from a cookie set by the client. Tenant claims must originate from an IdP assertion or from server-side configuration mapped to the IdP. Anything else is a trivial cross-tenant escalation vulnerability.

Where MFA actually happens in federation

In a federated setup, **the IdP owns the authentication strength**. When a tenant federates via Okta, it is Okta that enforces MFA, conditional access (device posture, geo, risk score), and password policy — not your platform. Your security review must therefore include verifying, both contractually and technically, that each federated tenant enforces phishing-resistant MFA and acceptable policy.

Practical controls your platform can layer on top:

- **Require strong ACR/AMR claims** — reject tokens whose `acr/amr` claims indicate weak authentication (e.g. password-only).
- **Broker-side advanced security** — Cognito and equivalents detect compromised credentials, impossible travel, and risky sign-ins as a second line.
- **Step-up authentication for sensitive actions** — require a fresh MFA challenge for high-risk operations regardless of the original session.
- **Session anomaly detection** — your application can detect unusual access patterns on top of whatever the IdP did.

5. Authorization models

Authentication answers “who.” Authorization answers “what may you do.” Five models, used in combination.

RBAC — Role-Based Access Control

Users are assigned roles, and roles carry permissions. Simple, well-understood, and the right answer for most coarse-grained decisions: “admins can manage users; analysts can run queries; viewers can see dashboards.” Implemented in your application's authorization layer, in Kubernetes RBAC, in cloud IAM, and in databases.

Where it breaks down: when permissions depend on context (which tenant's data, which time of day, which department), or when you have a combinatorial explosion of roles. A SaaS with 10,000 tenants cannot scale RBAC to a role per tenant.

ABAC — Attribute-Based Access Control

The authorization decision is a function of attributes — subject attributes (the caller's `tenant_id`, department, clearance), resource attributes (data classification, owning tenant, region), and environment attributes (time, IP, device posture). ABAC is what makes multi-tenant authorization scale: one policy can govern thousands of tenants.

```
// AWS IAM policy fragment - ABAC tag-match
{
  "Effect": "Allow",
  "Action": ["s3:GetObject", "s3:PutObject"],
  "Resource": "arn:aws:s3::analytics-data/*",
  "Condition": {
    "StringEquals": {
      "aws:PrincipalTag/tenant_id": "${aws:ResourceTag/tenant_id}"
    }
  }
}
```

This single policy enforces tenant isolation across an arbitrary number of tenants, structurally, at the IAM layer — not in application code. The trade-off is that **every resource must be correctly tagged** at creation, which becomes a hard tagging-discipline requirement enforced by policy-as-code in your IaC pipeline.

ReBAC — Relationship-Based Access Control

Permissions are defined by relationships: “user X is an editor of document Y,” “team Z owns folder F,” with permissions propagating across a relationship graph. This is the model behind Google's Zanzibar and open-source systems like SpiceDB and OpenFGA. Powerful for collaboration applications (file sharing, project management, document collaboration) but operationally more complex than RBAC or ABAC.

Policy-as-Code (PBAC) — OPA, Cedar, Rego

Externalize the authorization decision into a policy engine. Your application asks “can subject X do action Y on resource Z?” and the engine evaluates declarative policy and returns allow or deny. **OPA (Open Policy Agent)** is the de facto standard for Kubernetes admission control, microservice authorization, and CI/CD policy. **AWS Cedar** is the language behind AWS Verified Permissions, designed specifically for application-level authorization.

Why externalize: authorization logic stops being scattered across the codebase, becomes version-controlled, auditable, and testable like any other code. Policy can be reviewed independently from application logic; changes can go through a security review process; the same engine can enforce policy across many services.

Just-in-Time / Zero Standing Privilege

Applied especially to humans with privileged access — engineers, SREs, administrators. No one *has* production access by default; they *request* it for a defined window, the request is logged, approved (or auto-approved with conditions), and credentials expire when the window ends. Implemented via AWS IAM Identity Center session policies, HashiCorp Boundary, Teleport, or a workflow built on top of STS.

In a serious security program

No human has standing access to production data. Engineers debug via observability tools, not by querying production databases directly. When direct access is needed, it is requested, time-bound, justified, audited, and recorded. This is one of the few controls that genuinely defeats insider threat and post-compromise persistence.

6. Multi-cloud identity

How to run consistent identity across AWS, Azure, and Google Cloud without synchronizing user databases or storing static cross-cloud credentials.

The core problem

A workload in Google Kubernetes Engine needs to read data from an Amazon S3 bucket. An Azure Function needs to call a Google Cloud Pub/Sub topic. A CI/CD pipeline in GitHub deploys to all three clouds. None of these workloads should be storing long-lived access keys, and identity should not have to be redefined in each cloud.

The solution is **workload identity federation with OIDC trust**. Each workload has an identity native to where it runs; each target cloud is configured to trust that identity's issuer; credentials are exchanged short-term, on demand, with cryptographic verification at every step.

Federation hub patterns

Pattern	How it works	When to choose
One cloud as the identity plane	AWS / Azure / GCP IAM is the source of truth; other clouds federate to it via OIDC	Heavy investment in one cloud; secondary footprints are limited
Neutral identity provider as the hub	Okta, Auth0, Ping is the human identity hub; all clouds federate to it	Multi-cloud with significant human user access; existing IdP investment
Per-cloud identity with cross-cloud federation	Each cloud maintains its own identities; OIDC trust links them where workloads cross	Truly independent business units per cloud; clear data sovereignty boundaries
SPIFFE / SPIRE for workload identity	Universal workload identity (SPIFFE ID + SVID) regardless of cloud	Service-mesh-heavy architectures; cross-cloud workload-to-workload communication

Cross-cloud workload identity, by target

Workload runs in...	Calls AWS via...	Calls Azure via...	Calls GCP via...
AWS (EC2, EKS, Lambda)	Instance/role/IRSA — native	Azure AD federation; OIDC trust on Azure side	Workload Identity Federation — OIDC trust
Azure (VM, AKS, Functions)	OIDC trust + AssumeRoleWithWebIdentity	Managed identity — native	Workload Identity Federation — OIDC trust
GCP (GCE, GKE, Cloud Run)	OIDC trust + AssumeRoleWithWebIdentity	OIDC trust on Azure side	Service account / Workload Identity — native
Kubernetes (any cloud)	IRSA / OIDC + AssumeRoleWithWebIdentity	Azure Workload Identity (OIDC)	Workload Identity (OIDC)
GitHub Actions / CI	OIDC → AssumeRoleWithWebIdentity	OIDC → Azure federated credential	OIDC → Workload Identity Federation
On-premises / external	IAM Roles Anywhere (X.509)	Azure Arc — federated identity	Workload Identity Federation (X.509 or OIDC)

SPIFFE / SPIRE — universal workload identity

SPIFFE (Secure Production Identity Framework for Everyone) is a set of standards for issuing cryptographic identities to workloads regardless of where they run. SPIRE is the reference implementation that issues those identities. Each workload gets a **SPIFFE ID** (`spiffe://yoursaas.com/tenant-a/analytics-pod`) and a corresponding **SVID** — a signed verifiable identity document, either X.509 certificate or JWT.

Why it matters for multi-cloud: the same identity model applies to AWS EKS pods, GKE pods, AKS pods, and bare-metal services. You stop needing a different authentication mechanism for each cloud. Istio and Linkerd integrate natively. SPIFFE identities can be exchanged for cloud-specific credentials at the boundary, making it the cleanest answer to “how do we authenticate workloads consistently across clouds?”

The trade-off: SPIRE is operationally non-trivial — a control-plane server, agents on every node, trust-bundle management, and federation between SPIFFE trust domains for cross-cluster identity. Worth it for service-mesh-heavy, cross-cloud architectures; overkill for simpler estates.

Human identity across clouds

For human users — engineers, SREs, support staff — the cleanest model is a **single workforce IdP** federating to all three clouds:

- Workforce IdP (Okta, Entra ID, Google Workspace) is the source of truth for human identity, MFA, and conditional access.
- AWS IAM Identity Center, Azure RBAC (via Entra ID), and Google Cloud Identity each federate to the same workforce IdP.
- Engineers sign in once at the IdP and get short-lived, role-scoped sessions in each cloud through Just-in-Time elevation.
- Group membership in the IdP drives role assignments — add a user to the “Analytics-Engineering” group, and they automatically get the appropriate roles in all three clouds; remove them, and access vanishes everywhere.
- SCIM provisioning keeps cloud-side identity records in sync with IdP-side lifecycle events.

7. Tenant-aware identity in multi-tenant SaaS

How the `tenant_id` claim becomes the linchpin of multi-tenant isolation, and how to propagate it safely through every layer.

The `tenant_id` is sacred

In a multi-tenant SaaS, the `tenant_id` claim does the heavy lifting of isolation. Every authorization decision — from the API gateway to the database to the encryption key — ultimately depends on it. The claim must be:

- **Established by a trusted source** at authentication time — the IdP's assertion, or your broker's server-side mapping. Never derived from user input.
- **Carried as a verifiable claim** in every token throughout the system. The signature on the token protects the claim from tampering.
- **Re-checked at every authorization boundary** — not just at the front door. Defense in depth means the database, the storage layer, and the encryption layer each independently enforce the `tenant_id` match.
- **Audited on every access** — the `tenant_id` of the caller and the `tenant_id` of the resource are both in the audit log, so cross-tenant access attempts are detectable.

Claim propagation across the request path

```
User → IdP : authenticates, IdP signs assertion with tenant_id claim
IdP → Broker (Cognito) : assertion verified, broker mints JWT with tenant_id
Broker → App : JWT delivered to application
App → STS : app calls AssumeRoleWithWebIdentity with the JWT;
STS extracts tenant_id from the trusted claim into a
session tag (aws:PrincipalTag/tenant_id)
App → AWS APIs (S3, DB) : IAM evaluates resource policies that compare
PrincipalTag/tenant_id against the resource's tenant tag
App → internal services : tenant_id propagated in service-to-service tokens
(signed by a service mesh or workload identity)
App → database : query filtered by tenant_id; database also enforces RLS
App → KMS : per-tenant CMK; key policy checks caller's tenant tag
before allowing decrypt
```

Per-tenant identity boundaries

In addition to the `tenant_id` claim, the platform's identity infrastructure itself can be segregated per tenant for stronger isolation:

Approach	What's isolated	When to use
Single user pool, <code>tenant_id</code> claim	Logical (claim-based) isolation only	Direct-signup, smaller tenants, common case
One user pool per tenant	Separate user databases, MFA settings, app clients	Enterprise tenants who manage their own user lifecycle
Federated to tenant's IdP	Authentication entirely in tenant's control	Enterprise tenants with existing SSO; the default for large customers

Approach	What's isolated	When to use
Dedicated identity stack per tenant	Entire IAM scope is separate; cross-tenant access structurally impossible	Regulated, high-assurance, or sovereign-data tenants

8. Other methods worth knowing

Beyond the eight core flows: specialized patterns that earn their place in particular contexts.

SCIM — for the lifecycle, not the login

System for Cross-domain Identity Management. The protocol enterprise IdPs use to provision and de-provision users into your SaaS. Critical: when a tenant fires an employee, SCIM ensures that user is disabled in your platform within minutes, not weeks. Often overlooked in design, often the source of breaches involving departed employees who retained access through neglect.

What SCIM does: the tenant's IdP makes authenticated calls to your SCIM endpoint to create, update, disable, and delete users in your platform. Group memberships sync the same way, driving role assignments. The protocol is straightforward REST JSON; the operational discipline is the hard part — reliability, error handling, audit, and reconciliation when things drift.

CIBA — Client-Initiated Backchannel Authentication

An OIDC extension where the user authenticates via a separate channel — a push notification to their mobile app, a decoupled approval flow — rather than a browser redirect. Useful for kiosk applications, voice interfaces, IoT devices, and high-assurance approval flows where you want the approval to happen on a device the user already has authenticated.

Step-up authentication

The same user, the same session, but for high-risk actions (changing payment details, exporting all data, deleting a tenant), they are forced to re-authenticate with stronger factors. Implemented as `acr` (Authentication Context Class Reference) values in tokens; the application demands `acr=mfa-strong` for sensitive endpoints and triggers a fresh authentication if the current session does not meet the bar.

The pattern: not every action requires the same level of trust. A long-lived session is acceptable for reading dashboards; deleting an entire dataset deserves a fresh proof of identity. Step-up makes this graduated rather than all-or-nothing.

Decentralized identity / Verifiable Credentials

W3C standards for user-held identity: **Decentralized Identifiers (DIDs)** and **Verifiable Credentials (VCs)**. The user holds their own identity in a wallet and presents cryptographically verifiable claims to your application. Issuer signs a credential (“This person is a licensed physician”); the user holds it; the verifier (your app) checks the issuer's signature without phoning home.

Still emerging in mainstream enterprise SaaS, more established in regulated verticals (KYC, healthcare credentials, educational records). Worth watching but not yet a default — the ecosystem is fragmented and the user-experience problems (wallet recovery, cross-device sync) are not solved.

Network identity supplements

For B2B SaaS with strict enterprise tenants, network identity supplements user authentication: “only requests from this customer's IP range or via this PrivateLink endpoint are allowed to reach the authentication tier.” Not

authentication on its own — IPs spoof, networks move — but a strong additional factor and often a contractual requirement for regulated industries.

- **IP allow-listing** — the simplest, often demanded by financial-services and healthcare tenants.
- **PrivateLink / Private Endpoints / Private Service Connect** — the tenant's traffic reaches your service over the cloud provider's private backbone, never the internet.
- **Dedicated VPN or Direct Connect / ExpressRoute / Interconnect** — the strictest tenants; effectively a private circuit into your platform.
- **mTLS at the edge** — the client must present a certificate; combines network identity with cryptographic authentication.

API keys + HMAC request signing

Older pattern, still common for high-volume machine APIs. The client signs each request with a shared secret using HMAC-SHA256 over a canonical request representation. AWS's own SigV4 is the most famous example. Robust against replay attacks when combined with timestamps and nonces, but operationally heavy because of secret management. Use it when OAuth's token-handling overhead is genuinely too much (very high request rates with small payloads); otherwise prefer client credentials or workload identity federation.

9. Threat model and anti-patterns

What attackers target in identity systems, and the design mistakes that hand them victories.

What attackers actually target

Target	Attack	Primary mitigation
User credentials	Phishing, credential stuffing, breach reuse	Phishing-resistant MFA (passkeys); breach-password detection
Session tokens	Token theft via XSS, network capture, malicious extensions	HttpOnly cookies; short lifetimes; binding to client characteristics
Authorization code	Interception via referer leak, browser history, open-redirect on callback	PKCE; strict redirect-URI validation; short code lifetime
SAML/OIDC signature validation	Signature wrapping, key confusion, alg:none bypass	Vetted library; strict validation; reject weak algorithms
Client secrets	Theft from source code, CI logs, environment variables	Secret manager; rotation; ideally eliminate via workload identity
Refresh tokens	Long-term persistence after a single compromise	Rotation on use; revocation on anomaly; bound to client/device
The tenant_id claim	Manipulation in transit or at trust boundary	Source from IdP only; cryptographic signature on every token
IdP itself	IdP account takeover → unlimited access	Strong IdP-side controls; conditional access; out-of-band admin auth
Cross-tenant access logic	Bugs in application authorization code	ABAC in IAM, not just app code; defense in depth across layers

Anti-patterns to avoid

- **Trusting the client to assert tenant_id.** Any tenant identifier that originates in a header, URL parameter, or client-side state is forgeable. The tenant_id must come from the IdP or server-side mapping.
- **Storing tokens in localStorage.** Vulnerable to XSS exfiltration. Use HttpOnly cookies or a backend-for-frontend pattern.
- **Long-lived access tokens.** If an access token leaks and is valid for 24 hours, that's 24 hours of attacker access. Minutes-lived tokens with refresh rotation reduce the window dramatically.
- **Sharing client secrets across environments.** Production and staging should never share credentials. A compromised staging secret is a path into production.
- **Using ID tokens as API authorization.** ID tokens are for the client to learn user identity; access tokens are for API authorization. They have different audiences and different validation rules.
- **Authorization logic only in the application.** A single bug then defeats the entire isolation model. Defense in depth means authorization is also enforced at the IAM, database, and encryption layers.
- **Ignoring token replay.** Without nonces, jti tracking, or short windows, a captured token is reusable. Bind tokens to client characteristics where possible.

- **Forgetting de-provisioning.** Users leave companies; tokens persist; SCIM and session revocation must work and be tested.
- **Trusting one signal.** Single-factor authentication, single-layer authorization, single source of truth without verification — each is a single point of failure. Layer everything.
- **Inconsistent claims across services.** If `tenant_id` means different things in different services, your authorization model has a gap. Define the claim once; enforce its semantics everywhere.

10. Design principles and decision guide

Choosing among the flows; the principles that hold across all of them; the one-page summary.

Choosing an authentication flow

If your use case is...	Use this flow	Because
Enterprise tenant with their own IdP	SAML or OIDC federation	Tenant owns auth strength; SSO is a deal-breaker for enterprise
Direct-signup consumer/SMB tenant	OIDC Authorization Code + PKCE (broker)	Modern, secure, supports MFA fallback for non-federated users
Mobile or single-page application	OIDC Authorization Code + PKCE (mandatory)	Public clients; PKCE closes code-interception attacks
Server-to-server API call (no user)	OAuth 2.0 client credentials	Workload authentication; access token represents the service
CI/CD or workload calling cloud APIs	Workload Identity Federation	Zero stored credentials; short-lived; cryptographic trust
Pod-to-pod inside Kubernetes	Service-mesh mTLS (often SPIFFE-backed)	Authenticated and encrypted; automatic rotation
High-assurance human authentication	Passkeys (WebAuthn / FIDO2)	Phishing-resistant; no shared secret
Cross-cloud workload-to-workload	OIDC federation hub or SPIFFE	Same identity model across clouds; no static keys
High-risk action in an existing session	Step-up authentication (acr claim)	Graduated trust; fresh proof for sensitive actions

Principles that hold across every flow

- **Treat authentication and authorization as separate, equally rigorous concerns.** Federate authentication to the strongest IdP the tenant brings; enforce authorization in depth at every layer.
- **Federate everywhere; eliminate shared and static credentials.** Workload identity federation for machines; SSO and passkeys for humans. The goal is zero secrets stored anywhere.
- **Push authentication strength to the edge; push authorization enforcement to every layer.** Strong IdP-side authentication at the front door; multi-layer authorization in depth.
- **The tenant_id is sacred.** It comes from the IdP or a server-side mapping, never from user input. Once established, it propagates as a non-spoofable signed claim through every layer.
- **Short-lived everything.** Tokens minutes. Workload credentials minutes-to-hours. Refresh tokens rotated. Standing privilege replaced with just-in-time elevation.
- **Authentication should be cryptographic, not knowledge-based.** Passkeys over passwords; mTLS over API keys; OIDC trust over shared secrets.

- **Audit is part of the design, not an afterthought.** Every authentication event and every authorization decision is logged with full identity context, immutably, with the `tenant_id` of both caller and resource recorded. If the audit trail cannot reconstruct who-did-what-when, the system is not really secure — it is just hopeful.

The one-paragraph summary

A real multi-cloud SaaS uses several authentication flows in concert: OIDC federation for enterprise tenants, direct OAuth for SMB tenants, workload identity federation for service-to-service, mTLS in the service mesh, and passkeys for human MFA. These coexist, with consistent claims — especially the `tenant_id` — flowing through every flow into a single authorization model. The design discipline is making sure those claims mean the same thing everywhere, so that one authorization policy can govern the entire estate. The trap to avoid is choosing one method and forcing every use case through it; the discipline to maintain is that authentication strength belongs to the IdP, authorization enforcement belongs at every layer, and audit captures both.